

Praktikum 3

CampusManager mit Arrays

Ziele

Im dritten Praktikum vertiefen Sie den Umgang mit **Arrays** und **Methoden** und erweitern den CampusManager zu einem kleinen, konsolenbasierten Verwaltungssystem. Sie lernen, mehrere Module gleichzeitig zu speichern, auszuwerten und zu sortieren. Dazu setzen Sie systematisch um:

- das **Anlegen und Befüllen paralleler Arrays**,
- das **Auswerten und Aggregieren** von Daten über Schleifen,
- das **Arbeiten mit selbst definierten Methoden**,
- einfache **Sortierverfahren** am Beispiel eines vorgegebenen Algorithmus,
- den Aufbau eines **mehrteiligen Menüs** zur Navigation im Programm.

Das Praktikum bildet damit die Grundlage für die nächsten Erweiterungen im Projekt „CampusManager“ und bereitet auf zukünftige Themen wie komplexere Datenstrukturen, Objektorientierung und weitere Sortierverfahren vor.

Allgemeine Hinweise

- Das Praktikum 3 umfasst 3 Aufgaben, die als Hausaufgabe selbständig im 2er-Team gelöst werden sollen.
- Zum Bestehen des Praktikums müssen alle drei Aufgaben vollständig bearbeitet und funktionsfähig umgesetzt sein.
- Zum Praktikumstermin müssen die erstellten Programme fertig und lauffähig sein und von Ihnen vorgestellt werden.
- Im Git-Repository Ihres Teams finden Sie für dieses Praktikum bereits die drei Klassen `P3_A1_Semesteruebersicht`, `P3_A2_Sortieren` und `CampusManager` als Grundgerüst. Nutzen Sie die im Quelltext eingefügten Kommentare als Orientierung und ergänzen Sie dort Ihre eigene Lösung.
- Außerhalb der TH ist zwingend eine VPN-Verbindung zur TH notwendig, um auf Ihr Team-Git-Repository zugreifen zu können. Infos: https://www.th-koeln.de/hochschule/vpn---virtual-private-network_26952.php
- Abgabe:
 - Es ist eine gemeinsame Lösung pro Team abzugeben. Die Abgabe muss bis zum Abend vor dem Vorfürhrttermin, spätestens bis 18:00 Uhr, erfolgen.
 - Abgabeort: Push des veränderten Quellcodes zurück in Ihr Repository. Kontrollieren Sie über die Gitlab-Webseite, dass dies korrekt funktioniert hat.
 - Quellcode: muss kompilierbar und ausführbar sein.

Hinweise zur Bearbeitung

- Es dürfen **alle Inhalte verwendet werden**, die bis jetzt in der Vorlesung behandelt wurden:
 - Variablen
 - Kontrollstrukturen
 - Methoden
 - Arrays
 - Strings
 - Ein- und Ausgabe über die Konsole
- Sie dürfen und sollen die Eingabemethoden aus Praktikum 2 wiederverwenden:

```
import java.util.Scanner;

static final Scanner SC = new Scanner(System.in);

// Liest eine ganze Zeile ein (String)
static String readLine(String prompt) {
    System.out.print(prompt);
    return SC.nextLine();
}

// Liest eine ganze Zahl ein (int), inkl. Fehlerbehandlung
static int readInt(String prompt) {
    while (true) {
        try {
            return Integer.parseInt(readLine(prompt).trim());
        } catch (Exception e) {
            System.out.println("Bitte eine ganze Zahl eingeben.");
        }
    }
}

// Liest erstes Zeichen (char), als Großbuchstabe, bei leerer Eingabe '\n'
static char readChar(String prompt) {
    String s = readLine(prompt).trim();

    return s.isEmpty() ? '\n' : Character.toUpperCase(s.charAt(0));
}
```

Aufgabe 1 – Semesterübersicht eines Studierenden

In dieser Aufgabe speichern Sie mehrere Module eines Studierenden und berechnen daraus verschiedene Kennzahlen, die später im CampusManager verwendet werden.

Teil 1 – Module erfassen

- Fragen Sie zuerst die Anzahl der Module ab (1 ... 10).
- Legen Sie drei parallele Arrays an:

```
String[] modulName;  
int[]     modulects;  
int[]     modulNote;  
int       anzahlModule;
```
- Lesen Sie für jedes Modul ein:
 - Name (String)
 - ECTS (int ≥ 0)
 - Note (int 1..5)
- Ungültige Eingaben müssen jeweils erneut abgefragt werden.

Teil 2 – Auswertung durch Methoden

Schreiben Sie folgende Methoden:

- `static int gesamtEcts(int[] ects, int anzahl)`
→ Summe aller ECTS
- `static double durchschnittsnoteGewichtet(int[] noten, int[] ects, int anzahl)`
→ gewichtete Durchschnittsnote nach der Berechnung

$$\text{Note} = \frac{\sum(\text{note}[i] \cdot \text{ects}[i])}{\sum \text{ects}}$$

- `static int anzahlBestanden(int[] noten, int anzahl)`
→ Anzahl der bestandenen Module (Note ≤ 4)

Teil 3 – Ausgabe

Geben Sie aus:

- eine Tabelle aller Module
- die Gesamt-ECTS
- die gewichtete Durchschnittsnote
- die Anzahl bestandener Module

Tipp zu Aufgaben 1 und 2:

Legen Sie sich Beispiel-Arrays zum Testen Ihres Codes an! Dadurch müssen Sie nicht für jeden Test Daten per Konsole einlesen. Sie können folgende Arrays verwenden:

```
String[] modulName = {"MA1", "PH1", "DR", "PI1", "PI2", "PP",  
"PH1", "GE1", "PH2", "MA2"};
```

```
int[] modulEcts = {9, 5, 5, 5, 5, 3, 4, 5, 6, 9};
```

```
int[] modulNote = {1, 2, 3, 4, 5, 1, 2, 3, 4, 5};
```

Aufgabe 2 – Module sortieren

Für viele Auswertungen ist es hilfreich, Module sortieren zu können.
Dafür verwenden Sie die parallelen Arrays aus Aufgabe 1.

Vorbereiteter Sortieralgorithmus (BubbleSort)

Sie dürfen folgenden Algorithmus verwenden und müssen ihn **nur verstehen und anpassen**, aber **nicht neu erfinden**:

```
static void bubbleSortInt(int[] a, int n) {  
    boolean vertauscht;  
    do {  
        vertauscht = false;  
        for (int i = 0; i < n - 1; i++) {  
            if (a[i] > a[i + 1]) {  
                int tmp = a[i];  
                a[i] = a[i + 1];  
                a[i + 1] = tmp;  
                vertauscht = true;  
            }  
        }  
    } while (vertauscht);  
}
```

Ein Video zur Erläuterung des Algorithmus finden Sie z.B. hier:

<https://www.youtube.com/watch?v=gN6pDFOdGrE>

Aufgabe 2.1 – BubbleSort testen

- Legen Sie ein kleines Beispielarray an (z. B. {5, 2, 9, 1, 3}).
- Testen Sie bubbleSortInt und geben Sie vorher/nachher aus

Aufgabe 2.2 – Module nach Note sortieren (Pflicht)

Schreiben Sie die Methode

```
static void sortiereNachNote(String[] namen, int[] ects, int[]  
noten, int anzahl)
```

- Sortieren Sie die Module aufsteigend nach Note und geben Sie eine Tabelle aller Module aus. (Das Ausgeben können Sie aus Aufgabe 1 übernehmen.)

Wenn `noten[i]` und `noten[j]` getauscht werden, müssen alle drei Arrays getauscht werden.

→ Nutzen Sie die BubbleSort-Struktur als Vorlage!

Aufgabe 2.3 – Weitere Sortierungen (optional)

(a) Sortieren nach ECTS (absteigend)

Hinweis: Vergleichsoperator umdrehen (< statt >).

(b) Sortieren nach Namen (alphabetisch)

Hinweis:

BubbleSort muss hierfür mit einem String-Vergleich angepasst werden:

```
if (namen[i].compareTo(namen[i+1]) > 0) {  
    // tauschen  
}
```

Alle drei Arrays müssen wieder gemeinsam getauscht werden.

Aufgabe 3 – CampusManager 2.0

Dies ist die Hauptaufgabe des Praktikums.

Sie implementieren ein konsolenbasiertes System, das mehrere Module verwaltet.

Datenstrukturen

Verwenden Sie parallele Arrays (max. 20 Module):

```
String[] modulName  = new String[20];  
int[]      modulEcts  = new int[20];  
int[]      modulNote  = new int[20];  
  
int anzahlModule = 0; // Anzahl aktuell gespeicherter Module
```

Menü

Ihr Programm soll wiederholt folgendes Menü anzeigen, bis der Benutzer 0 eingibt:

- 1 - Modul hinzufügen
- 2 - Alle Module anzeigen
- 3 - Modul löschen
- 4 - Statistik anzeigen
- 5 - Notenänderung (optional)
- 6 - Sortieren (optional)
- 0 - Beenden

Ungültige Eingaben führen zu einer Fehlermeldung und das Menü wird erneut angezeigt.

Funktionen

Schreiben Sie für jeden Menüpunkt eine eigene Methode:

- `modulHinzufuegen`
 - Name, ECTS (≥ 0) und Note (1..5) einlesen.
 - Gültigkeit prüfen, Modul speichern, maximale Modulanzahl beachten.
- `moduleAnzeigen`
 - Alle aktuell gespeicherten Module übersichtlich ausgeben.
- `modulLoeschen`
 - Modul anhand einer gewählten Modulnummer entfernen.
 - Daten so anpassen, dass keine Lücke in den Arrays entsteht (Verschieben).
 - Bei ungültiger Modulnummer eine Fehlermeldung ausgeben.
- `statistik`
 - Gesamt-ECTS, Anzahl bestandener Module, beste und schlechteste Note, gewichtete Durchschnittsnote ausgeben.
 - Methoden aus Aufgabe 1 können wiederverwendet werden.

optional:

- `noteAendern`
 - Modul über Modulnummer auswählen und Note (1..5) ändern.
 - Bei ungültiger Modulnummer eine Fehlermeldung ausgeben.
- `sortieren`
 - Module nach einem Kriterium (z. B. Note) sortieren und ausgeben.
 - Nutzen Sie hierfür Ihre Lösungen aus Aufgabe 2.